Shengkai Xu
ECGR 4090 – Real time AI
03/9/2021
Homework 2

GitHub: https://github.com/ric3b0wl/ECGR4090-RealTimeAI/hw2

Problem 1 "Temp" Tensor from list(range(9))

https://github.com/ric3b0wl/ECGR4090-RealTimeAI/blob/main/hw2/prob1.ipynb

Improvements

Python API

- Add integer support (by promoting integer to float) to `torch.{cos,sin,tan}` (#45733, #46706), `torch.log{2,10}` (#46810), `torch.{a}tanh` (#47064), `torch.a{cos, tan}` (#47005), `torch.a{cosh, sinh}` (#47152), `torch.sqrt` (#47293), `torch.log1p` (#48002). `torch.erf{c}` (#48472), `torch.asin` (#48461), `torch.sigmoid` (#47551), `torch.sinh` (#48644), `torch.cosh` (#48923), `torch.exp{2, m1}` (#48926), `torch.reciprocal` (#49102), `torch.erfinv` (#49155), `torch.rsqrt` (#47909), `torch.exp` (#50093), `torch.lgamma` (#50140)

In the new Pytorch update, it improved the support of integer with cos and sqrt function. In the old version I believe the tensor must be a float value to be able to use torch. sqrt() and torch.cos() to get a valid result.

```
In [ ]: import torch

        # temp = torch.tensor(list(range(9)))
        temp = torch.Tensor(list(range(9)))
        print(f'{temp.type()}')
        temp

        torch.FloatTensor
Out[ ]: tensor([0., 1., 2., 3., 4., 5., 6., 7., 8.])

In [ ]: temp.sqrt()
Out[ ]: tensor([0.0000, 1.0000, 1.4142, 1.7321, 2.0000, 2.2361, 2.4495, 2.6458, 2.8284])

In [ ]: temp.cos()
Out[ ]: tensor([ 1.0000,  0.5403, -0.4161, -0.9900, -0.6536,  0.2837,  0.9602,  0.7539,
                -0.1455])
```

Problem 2 Red, Blue, Green image RGB brightness

https://github.com/ric3b0wl/ECGR4090-RealTimeAI/blob/main/hw2/prob2.ipynb

```
In [81]: from google.colab import drive
         drive.mount('/content/drive')

         # from skimage.transform import resize
         # from skimage.color import rgb2gray
         import matplotlib.pyplot as plt
         import numpy as np
         import torch
         import imageio


         device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

         Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

In [82]: red_dir = "/content/drive/My Drive/ECGR4090-RealTimeAI/hw2/red.jpg"
         green_dir = "/content/drive/My Drive/ECGR4090-RealTimeAI/hw2/green.jpg"
         blue_dir = "/content/drive/My Drive/ECGR4090-RealTimeAI/hw2/blue.jpg"
```

In [83]:
```python
red_img = imageio.imread(red_dir)
print("NumPy: ", type(red_img))
print("Shape: ", red_img.shape)
print("dtype: ", red_img.dtype)

red_img_np = torch.from_numpy(red_img)
print('Mean value: ', torch.mean(red_img_np.float()) )

print(red_img)
```

```
NumPy:  <class 'imageio.core.util.Array'>
Shape:  (244, 432, 3)
dtype:  uint8
Mean value:  tensor(136.5813)
[[[206 222 245]
  [207 223 246]
  [207 223 246]
  ...
  [167 209 247]
  [168 210 248]
  [167 211 248]]

 [[206 222 245]
  [206 222 245]
  [206 222 245]
  ...
  [162 206 243]
  [161 205 242]
  [161 205 242]]

 [[206 222 245]
  [206 222 245]
  [206 222 245]
  ...
  [156 201 240]
  [156 201 240]
  [155 200 239]]

 ...

 [[114 113 118]
  [115 114 120]
  [115 114 120]
  ...
  [127 123 124]
  [131 127 128]
  [128 124 125]]

 [[114 113 118]
  [117 116 121]
  [120 119 125]
  ...
  [128 124 125]
  [131 127 128]
  [129 125 126]]

 [[126 125 130]
  [130 129 134]
  [133 132 137]
  ...
  [128 124 125]
  [130 126 127]
  [134 130 131]]]
```

I was able to convert image to tensor and tensor come with three channels represent the RGB color for image. Getting the average value of image tells the brightness of the image. When getting average value of the channel determents the dominating color of the image. An image tensor is size [3,x,y] where [0,:,:] is red pixels, [1,:,:] is blue pixels, and [2,:,:] is green pixels.